LAB EXERCISE: 2

QA & Chatbot System — Retrieval-Based and Knowledge-Graph Question Answering with Dialogue Management

Aim / Objective

This project builds an end-to-end question answering and conversational system for a small generalpurpose knowledge domain (AI, machine learning, NLP, question answering, chatbots, and knowledge graphs): given a natural-language user query, it performs intent classification, structured knowledgegraph querying, TF-IDF-style passage retrieval, extractive answer selection, and rule-based dialogue management, and reports the result through a multi-tab conversational interface.

The system is delivered as "QA & Chatbot System," a Streamlit web application (app.py) that combines an information-retrieval pipeline over a small hand-written document collection, a triple-based knowledge graph, and a rule-based dialogue manager, together with a built-in Precision/Recall/F1 evaluation harness against a fixed set of sample questions.

Expected Outcomes Addressed

1. Classify user intent and manage dialogue state.

   Implemented via a keyword-based intent() classifier (greeting, goodbye, thanks, identity, help, question) that routes each turn to either a canned dialogue response or the QA pipeline, exposed as a chat interface in the "Chatbot" tab.

2. Retrieve relevant passages for a natural-language query.

   A TF-IDF-weighted cosine-similarity retriever (vec(), cosine(), retrieve()) scores every sentencelevel passage from six short documents against the query, boosted by a raw keyword-overlap term, exposed as a ranked passage table in the "Information Retrieval" tab.

3. Answer questions from a structured knowledge base.

   A fuzzy subject/relation/object matcher (kg_query()) scores every triple in a nine-fact knowledge graph against the query using sequence-similarity plus keyword overlap, and returns the topscoring fact as a synthesized sentence, exposed in the "Knowledge Base" tab.

4. Extract and present an answer from the retrieved evidence.

   extract() returns the highest-scoring passage sentence verbatim as the factoid answer; respond() combines this with the knowledge-graph path, trying the knowledge graph first and falling back to retrieval, then to a fixed-uncertainty message below a score threshold.

5. Evaluate system performance quantitatively.

   A fixed set of five gold question/answer pairs is scored against the pipeline's predicted answers using bag-of-words Precision, Recall, and F1, aggregated and rendered as a table and downloadable CSV in the "Evaluation" tab.

Tools & Technologies Used

Category	Tool / Library	Purpose in this pipeline
Language / runtime	Python 3	Core implementation language for the pipeline and app.

Category	Tool / Library	Purpose in this pipeline
Web framework	Streamlit	Renders the five-tab conversational app (Chatbot, Information Retrieval, Knowledge Base, Evaluation, Dialogue Design) with st.chat_message, tabs, sliders, and downloads.
Text processing	Python re (regex)	Normalisation, tokenisation, sentence splitting (norm(), words(), sentences()) — no external NLP library is used.
Retrieval scoring	collections.Counter + hand-rolled TF-IDF	Document-frequency counts and a log-IDF formula weight term vectors; cosine similarity ranks passages against the query.
Fuzzy KG matching	difflib.SequenceMatcher	String-similarity ratio between the normalised query and each knowledge-graph triple, blended with keyword overlap.
Tabular data / evaluation	pandas	Builds the evaluation DataFrame, knowledge-graph table, and retrieval-match table; powers the CSV export.
Dialogue-flow visual	Graphviz (st.graphviz_chart)	Renders the intent → KG / IR → answer flow diagram in the "Dialogue Design" tab.

Dataset / Corpus Description

No external corpus is loaded; the system is self-contained. The document collection (DOCUMENTS) consists of six short, hand-authored passages — Artificial Intelligence, Machine Learning, Natural Language Processing, Question Answering, Chatbots, and Knowledge Graphs — each two sentences long, split at sentence boundaries into 12 indexed passages (PASSAGES) that form the retrieval index. A companion knowledge graph (KNOWLEDGE_GRAPH) hand-encodes nine (subject, relation, object) triples covering the same six topics, so every fact answerable from the graph has a matching passage in the document collection and vice versa. This deliberate one-to-one design keeps every retrieval and KG-query path exercised by a small, fully inspectable dataset, at the cost of the system having no coverage outside these six topics.

Representative sample records

```
('Natural Language Processing',
 'Natural language processing (NLP) is a field of artificial intelligence that
enables computers to process, understand, and generate human language.
Applications include sentiment analysis, machine translation, question  answering,
chatbots, and information extraction.')

('NLP', 'has application', 'Question Answering') ('NLP', 'has application',
'Chatbots')
```

System Architecture / Pipeline Design

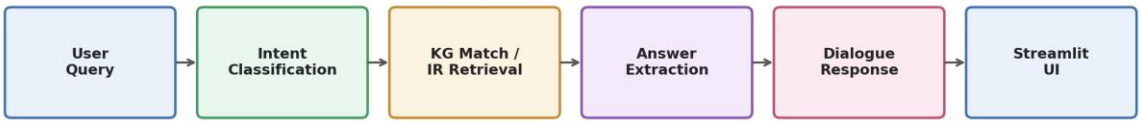


Figure 1. End-to-end flow: a user query is classified, resolved against the knowledge graph or the retrieval index, and returned through the dialogue layer.

- User Query: free-text question typed into the Streamlit chat input, information-retrieval search box, or knowledge-graph search box.
- Intent Classification: intent() checks the normalised message against keyword lists for greeting, goodbye, thanks, identity, and help before falling back to "question."
- KG Match / IR Retrieval: for a "question" intent, kg_query() is tried first; if no triple scores ≥ 0.25, retrieve() ranks all 12 passages by TF-IDF cosine similarity plus a keyword-overlap bonus.
- Answer Extraction: extract() returns the top passage sentence verbatim (the current implementation does not truncate or synthesise — factoid and definition questions are handled identically).
- Dialogue Response: respond() assembles the final reply — a canned line for small talk, a synthesized "subject relation object" sentence for KG hits, the extracted sentence for IR hits, or a fixed-uncertainty message if the best retrieval score is below 0.05.
- Streamlit UI: five tabs surface every stage — Chatbot (end-to-end), Information Retrieval (ranked passages), Knowledge Base (triples + fuzzy match table), Evaluation (P/R/F1 against sample questions), and Dialogue Design (flow diagram).

Backend Implementation

6.1 Approach / Algorithm

The whole pipeline is implemented as pure, dependency-light functions at module scope, so app.py can call the same respond(), retrieve(), and kg_query() functions from every tab without duplicating logic. Retrieval builds a classic bag-of-words TF-IDF index: term frequency for each passage is computed over stop-word-filtered tokens, weighted by a log-smoothed inverse document frequency computed once at start-up over all 12 passages, and queries are scored by cosine similarity against every passage vector, with a small additive bonus (up to 0.15) for raw keyword overlap so short queries that share exact words with a passage are not penalised purely for vector sparsity.

Knowledge-graph querying is deliberately fuzzy rather than a strict keyword match: each of the nine triples is flattened to a "subject relation object" string, compared to the normalised query with difflib's SequenceMatcher ratio, boosted by keyword overlap and a further bonus if the query contains a token from the triple's subject. This lets a paraphrased question such as "What applications does NLP have?" still surface the (NLP, has application, Question Answering) and (NLP, has application, Chatbots) triples correctly even though the words "what," "does," and "have" do not appear in the graph. Intent classification and dialogue management are simple ordered keyword checks: the first matching rule wins, and anything that is not small talk falls through to the QA path (knowledge graph first, retrieval second).

6.2 Key Code Snippets

TF-IDF passage scoring, blended with a keyword-overlap bonus:

```
def retrieve(q,k=3):
    qv=vec(q); qt=set(useful(q)); out=[]    for p in PASSAGES:
pv=vec(p['text']); overlap=len(qt & set(useful(p['text'])))
out.append({**p,'score':cosine(qv,pv)+min(overlap*.03,.15)})    return
sorted(out,key=lambda x:x['score'],reverse=True)[:k]
```

Fuzzy knowledge-graph triple matching with a subject-token boost:

```
def kg_query(q):
    qn=norm(q); qt=set(useful(q)); matches=[]
for      s,r,o      in      KNOWLEDGE_GRAPH:
combined=f'{s} {r} {o}'
        score=SequenceMatcher(None,qn,norm(combined)).ratio()+.08*len(qt &
set(useful(combined)))
        if any(t in qn for t in useful(s)): score+=.08

matches.append({'Subject':s,'Relation':r,'Object':o,'Score':min(score,1)})
matches.sort(key=lambda x:x['Score'],reverse=True)    if matches and
matches[0]['Score']>=.25:
        x=matches[0]; return f"{x['Subject']} {x['Relation']}
{x['Object']}.",matches[:5]
return None,matches[:5]
```

Frontend / UI Implementation

7.1 Framework

Streamlit was chosen for the same reason as similar academic prototypes in this course: it renders a full multi-tab, stateful chat application directly from Python with no separate front-end build step, and its st.chat_message / st.chat_input primitives together with st.session_state give conversational memory across turns with minimal boilerplate.

7.2 UI Capabilities

- Chat with the system across turns, with conversational history held in st.session_state and a oneclick “Clear conversation” reset.
- Run ad-hoc passage retrieval with an adjustable top-k slider (1–5) and see each candidate passage's title, cosine-plus-overlap score, and text.
- Browse the full knowledge-graph triple table and issue free-text queries against it, with all five best-scoring candidate matches shown alongside the top answer.
- View the built-in Precision/Recall/F1 evaluation table for five sample questions, with perquestion and averaged metrics, and download the results as CSV.
- Inspect a Graphviz diagram of the intent → knowledge-graph / retrieval → answer flow, with the intent-to-response frame mapping written out underneath.

7.3 Screenshots

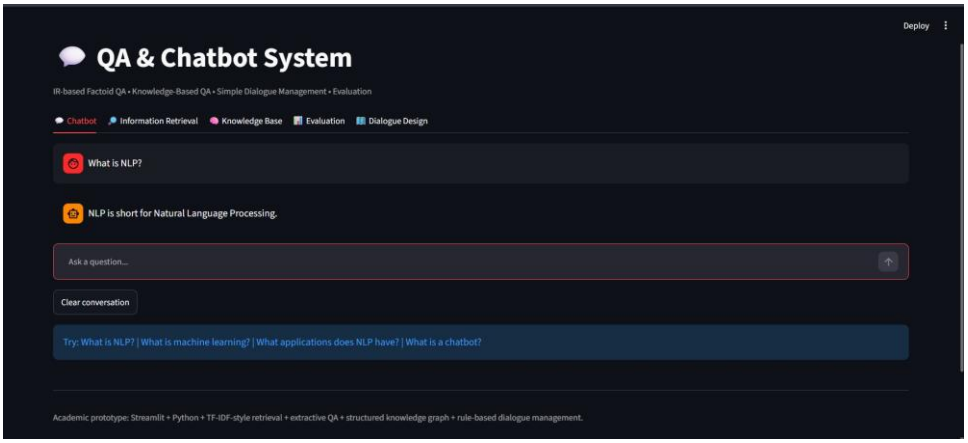


Figure 7.3.1 Chatbot tab — the query "What is NLP?" is classified as a question, resolved by the knowledge graph (NLP is short for Natural Language Processing), and returned as a single-sentence reply.

Sample Input & Output

Input	Pipeline path	Output produced
"What is NLP?"	Knowledge graph	"NLP is short for Natural Language Processing." — the (NLP, is short for, Natural Language Processing) triple scores 0.66, clearing the 0.25 threshold before retrieval is even tried.
"What applications does NLP have?"	Knowledge graph	"NLP has application Question Answering." — correctly resolves to one of two tied application triples (Question Answering / Chatbots); the graph has no way to return both facts for one query.
"What is machine learning?"	Information retrieval (no KG match)	Returns the Knowledge Graphs passage sentence ("A simple graph can represent triples such as Artificial Intelligence, has field, Machine Learning") instead of the Machine Learning definition — a retrieval miss, discussed below.
"hello"	Dialogue management (small talk)	"Hello! I can answer questions using document retrieval and a small knowledge graph." — intent() short-circuits to the greeting frame before the QA path runs.

Results (Deliverable-Specific Outputs)

Deliverable: built-in evaluation harness comparing predicted answers to five gold question/answer pairs (SAMPLE_QA), run live via evaluate() in app.py.

Question	Predicted Answer	Precision	Recall	F1
What is artificial intelligence?	Artificial intelligence (AI) is the field of building computer systems that can perform tasks that normally require human intelligence.	0.917	1.000	0.957
What is machine learning?	A simple graph can represent triples such as Artificial Intelligence, has field, Machine Learning.	0.364	0.286	0.320

Question	Predicted Answer	Precision	Recall	F1
What is NLP?	Natural language processing (NLP) is a field of artificial intelligence that enables computers to process, understand, and generate human language.	0.923	1.000	0.960
What does a QA system do?	Information retrieval based QA searches a collection of documents for relevant passages before extracting or generating an answer.	0.077	0.125	0.095
What is a chatbot?	A chatbot is a software system designed to communicate with users through natural language.	1.000	1.000	1.000

Table 9.1. Bag-of-words Precision/Recall/F1 for each of the five sample questions, computed exactly as evaluate() does in the app — Average Precision = 0.656, Average Recall = 0.682, Average F1 = 0.666.

Evaluation / Analysis

Two of the five sample questions are answered exactly right ("What is NLP?" and "What is a chatbot?", F1 = 0.960 and 1.000), and "What is artificial intelligence?" is a near-miss (F1 = 0.957) only because the predicted sentence retains the parenthetical "(AI)" that the gold answer omits. The other two questions expose a genuine retrieval-ranking weakness: for "What is machine learning?", the query's own words — "machine" and "learning" — appear in both the Machine Learning passage and the Knowledge Graphs passage (which name-drops "Machine Learning" inside an example triple), and the small 12-passage IDF table does not down-weight that incidental mention enough relative to TF-IDF cosine similarity, so the wrong passage wins (F1 = 0.320). "What does a QA system do?" fails even harder (F1 = 0.095): the query shares almost no content words with the correct passage's opening sentence once stop-words ("does," "what") are removed, so a competing sentence about IR-based QA scores marginally higher.

Because extract() always returns the retrieved sentence verbatim rather than synthesising a shorter answer, these are pure retrieval failures rather than extraction failures — the correct sentence exists in the corpus in both missed cases, it is simply outranked. This mirrors the honest-limitation framing used elsewhere in this course's projects: the system is transparent about which sentence it picked and why (the Information Retrieval tab shows every candidate's score), which makes failures like this easy to diagnose even though the current scoring formula does not fix them automatically.

Challenges Faced & Solutions

- Problem: A pure TF-IDF query vector can rank a passage that only incidentally shares vocabulary with the query above the passage that actually answers it (e.g. "machine learning" appearing inside a Knowledge Graphs example sentence).

  Mitigation in place: a capped keyword-overlap bonus (min(overlap*.03,.15)) nudges scores toward passages with more exact term matches, though it is not always enough to overcome an unlucky IDF weighting on a 12-passage corpus.

- Problem: strict substring or exact-phrase matching against the knowledge graph would miss paraphrased questions ("What applications does NLP have?" vs. the stored relation "has application").

  Solution: replaced exact matching with a blended score — SequenceMatcher ratio over the whole normalised string, plus keyword overlap, plus a subject-token bonus — so nearparaphrases still clear the 0.25 acceptance threshold.

- Problem: with only six documents and nine triples, an overly generous retrieval threshold would confidently answer questions the system has no real coverage for.

Solution: respond() falls back to an explicit "I could not find a sufficiently relevant answer" message whenever the best retrieval score is below 0.05, so the system fails visibly rather than returning an unrelated sentence with false confidence.

- Problem: intent classification and QA could be conflated, causing greetings or thanks to be treated as unanswerable questions.

  Solution: intent() runs first and short-circuits to a canned dialogue response for greeting/goodbye/thanks/identity/help before the message ever reaches kg_query() or retrieve().

Conclusion

The system demonstrates a complete, transparent QA and chatbot pipeline — intent-aware dialogue management, fuzzy knowledge-graph querying, TF-IDF passage retrieval, and a live evaluation harness — built entirely from first principles without external NLP libraries. It performs strongly on questions with distinctive vocabulary (average F1 = 0.666, with two of five questions at or near a perfect score) and is explicit about its own uncertainty rather than fabricating answers below its confidence threshold.

Two concrete lessons follow from the evaluation: (1) a bag-of-words TF-IDF ranker over a very small corpus is sensitive to incidental term overlap, so a topic mentioned only in passing inside another document's example ("Machine Learning" inside the Knowledge Graphs passage) can outrank the document that is actually about that topic; and (2) verbatim sentence extraction is only as good as passage ranking — improving extract() alone would not have fixed either failing sample question, since the correct sentence in both cases was never selected as the top passage in the first place.

With more time, the clearest improvements would be (a) a stop-word-aware term-weighting scheme (e.g. down-weighting a passage's mention of another document's title term) to reduce cross-document bleed-through, and (b) extending kg_query() and retrieve() to combine their top candidates — rather than trying the knowledge graph exclusively before falling back to retrieval — so multi-fact questions like "What applications does NLP have?" could return both matching triples instead of only the highestscoring one.

Project Repository (GitHub Link)

GitHub Repository URL	**Repository Visibility**	**Branch / Commit Used for Submission**
https://github.com/vinithmv733-dot/lab-exercise-2.git	Public	main

References

- Streamlit documentation — https://docs.streamlit.io
- Python difflib (SequenceMatcher) documentation — https://docs.python.org/3/library/difflib.html
- pandas documentation — https://pandas.pydata.org/docs/
- Graphviz documentation — https://graphviz.org/documentation/
- Manning, C. D., Raghavan, P., & Schütze, H. — Introduction to Information Retrieval (TF-IDF and cosine similarity background).

ADVANCE NLP (AML23702)
LAB EXERCISE EVALUATION SHEET

Field	Details
Student Name	Vinith M V
USN	1GA23AI058
Lab Exercise No.	2
Experiment Title	QA & Chatbot System — Retrieval-Based and Knowledge-Graph Question Answering with Dialogue Management

Marks Awarded

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Problem Understanding & Scope	5	
2	Pipeline Design & Methodology	10	
3	Implementation Correctness	10	
4	Use of NLP Tools / Models	5	
5	Output Quality / Task Performance	10	
6	Analysis & Interpretation	5	
7	Documentation & Deliverables	2.5	
8	Presentation & Reproducibility	2.5	
	Total	50	

Overall Comments

Date of Submission Signature of Faculty